

EventFinder - JavaScript Files Documentation

This document provides a comprehensive explanation of all JavaScript files in the EventFinder project, including all functions, their purposes, and how they work together.

Table of Contents

- 1. [modal1.js](#) - Core Modal System
- 2. [valid1.js](#) - Form Validation
- 3. [toggle_modes.js](#) - Theme Management
- 4. [both_homepage.js](#) - Shared Homepage Functions
- 5. [homepage_only.js](#) - Public Landing Page
- 6. [admin_dashboard.js](#) - Admin Functions
- 7. [student_dashboard.js](#) - Student Functions
- 8. [history_handler.js](#) - History Modal System

modal1.js

File Location: `src/js/modal1.js`
Purpose: Generic modal loading and management system
Dependencies: None
Used by: All dashboard pages for dynamic content loading

Core Functions

`openModal(htmlPath, modalId, onReady)`

Opens a modal dialog by dynamically loading HTML content.

Parameters:

- `htmlPath` (string) - Path to HTML file containing `.modal-content` div
- `modalId` (string) - Unique ID for modal container
- `onReady` (function) - Callback executed after modal loads and injects content

What it does:

- 1. Creates or reuses div with given modalId
- 2. Fetches HTML from htmlPath
- 3. Extracts `.modal-content` from fetched HTML using DOMParser
- 4. Injects content into modal div
- 5. Adds 'show' class to display modal
- 6. Hides page overflow (prevents scrolling)
- 7. Executes any `<script>` tags in loaded HTML
- 8. Calls onReady callback function

Example Usage:

```
openModal('../frontends/admin/create_event.html', 'createEventModal', function(modal) {
    // Modal is now open and ready to use
    console.log('Modal loaded:', modal);
});
```

Key Features:

- Reuses existing modals if already loaded
- Properly executes scripts in loaded HTML
- Handles fetch errors with user alert
- Manages page scroll state

`closeModal(modalId)`

Closes and clears a modal dialog.

Parameters:

- `modalId` (string) - ID of modal to close

What it does:

- 1. Finds modal by ID

2. Removes 'show' class (triggers CSS hide)
3. Re-enables page scroll
4. Clears modal innerHTML (fresh load next time)

Example Usage:

```
closeModal('createEventModal'); // Closes the modal
```

How It Works Together

The modal system follows this flow:

```
User clicks button
→ JavaScript calls openModal()
→ Fetches HTML file
→ Parses and extracts .modal-content
→ Injects into DOM
→ Shows overlay
→ Executes embedded scripts
→ Calls onReady callback for additional setup
```

When form submitted or close clicked:

```
closeModal() called
→ Hides modal with CSS
→ Clears content (prevents memory leaks)
```

valid1.js

File Location: src/js/valid1.js

Purpose: Real-time form validation for signup and login pages

Dependencies: None

Used by: login.html, signup.html

Regex Patterns (Module-Level)

```
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#%&*()_+~\-=\[\]\{\};':"\|,.<>\/?]).{8,}$/;
const collegeEmailRegex = /^[a-zA-Z]+\d+(bit|bca)\d+@kcc\.edu\.np$/i;
const phoneRegex = /^(98|97)\d{8}$/;
```

Pattern Explanations:

- passwordRegex - 8+ chars, uppercase, lowercase, digit, special char
- collegeEmailRegex - Format: name1bit2@kcc.edu.np (KCC college email only)
- phoneRegex - Nepal phone: starts with 98 or 97, followed by 8 digits

Helper Functions

isCollegeEmail(email)

Checks if email matches KCC college email format.

```
function isCollegeEmail(email) {
    return collegeEmailRegex.test(email);
}
```

Returns: true if college email, false otherwise

isValidEmail(email)

Basic email format validation (not college-specific).

```
function isValidEmail(email) {  
    return /^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/\\.test(email);  
}
```

Returns: true if valid email format, false otherwise

isValidPassword(pw)

Checks password strength requirements.

```
function isValidPassword(pw) {  
    return passwordRegex.test(pw);  
}
```

Returns: true if password meets all requirements

isValidPhone(phone)

Validates Nepal phone number format.

```
function isValidPhone(phone) {  
    return phoneRegex.test(phone);  
}
```

Returns: true if valid Nepal phone format

Field Validation Functions

validateFullname(val)

Validates full name field.

Rules:

- Required (not empty)
- Minimum 2 characters
- Only letters, spaces, hyphens, apostrophes

Returns: Error message string (empty string = valid)

```
validateFullname('John O'Brien') // Returns: ''  
validateFullname('J') // Returns: 'Name must be at least 2 characters.'  
validateFullname('123') // Returns: 'Name can only contain letters, spaces...'
```

validateEmail(val)

Validates email with college email warning.

Rules:

- Required
- Valid email format
- College email preferred (but not required)

Returns: Object with { error, info }:

- error - Blocks form submission if set
- info - Warning message if non-college email

```
const result = validateEmail('john@kcc.edu.np');
// Returns: { error: '', info: '' }

const result2 = validateEmail('john@gmail.com');
// Returns: { error: '', info: 'admin approval needed.' }
```

validateAddress(val)

Validates residential address.

Rules:

- Required
- Minimum 5 characters

Returns: Error message string

validatePhone(val)

Validates phone number.

Rules:

- Required
- Must be valid Nepal phone format (98/97 + 8 digits)

Returns: Error message string with format example

validatePassword(val)

Validates password strength.

Rules:

- Minimum 8 characters
- At least 1 uppercase letter
- At least 1 lowercase letter
- At least 1 digit
- At least 1 special character

Returns: Error message for first failed requirement

```
validatePassword('Pass123!') // Returns: ''
validatePassword('Pass') // Returns: 'Password must be at least 8 characters.'
validatePassword('password123!') // Returns: 'Add at least one uppercase letter.'
```

validateConfirmPassword(val, pw)

Validates password confirmation.

Rules:

- Required
- Must match password field

Parameters:

- `val` - Confirm password field value
- `pw` - Original password field value

Returns: Error message string

UI Feedback Functions

setFeedback(fieldName, msg, level = 'error')

Shows or clears validation feedback under form field.

Parameters:

- `fieldName` - Field identifier (matches `data-feedback` attribute)
- `msg` - Message to display (null/empty = clear)
- `level` - 'error' or 'info' (affects styling)

What it does:

1. Finds element with `[data-feedback=fieldName]`
2. Sets text content to message
3. Applies CSS class based on level (error = red, info = yellow)
4. Empty message clears the feedback

Example HTML:

```
<input id="email" type="email">
<p data-feedback="email"></p>

<!-- JavaScript: -->
<script>
setFeedback('email', 'Invalid email format', 'error');
// Shows red text under input
</script>
```

setInputState(input, hasError)

Adds visual indicator (border color) to input field.

Parameters:

- `input` - Input element
- `hasError` - Boolean, true for error state

What it does:

1. Adds 'input-error' class if hasError true (red border)
2. Adds 'input-success' class if hasError false (green border)

Form State Management

updateSubmitButton()

Enables/disables signup submit button based on form validity.

What it does:

1. Gets all form field values
2. Runs each validator
3. Disables button if ANY field has error
4. Enables button only when ALL fields valid

Logic:

```
const allValid =
  !validateFullname(fullname) &&
  !validateEmail(email).error &&
  isValidEmail(email) &&
  // ... other checks

btn.disabled = !allValid;
```

Non-College Email Modal

showNonCollegeModal(email, onProceed)

Shows modal for non-college email confirmation.

Parameters:

- `email` - The non-college email entered
- `onProceed` - Callback function if user proceeds

What it does:

1. Creates overlay modal with warning message
2. Shows email entered
3. Explains admin approval required
4. Two buttons: "Wait for Approval" / "Use Different Email"
5. If approve: sets `nonCollegeAcknowledged = true`, calls `onProceed`
6. If cancel: removes modal, focuses email input for correction

Module Variable:

```
let nonCollegeAcknowledged = false;  
// Set to true when user confirms non-college email
```

Form Submission Handlers

`handleLoginSubmit(event)`

Validates login form before submission.

What it does:

1. Checks email and password not empty
 2. Shows alerts for missing fields
 3. Returns false to prevent submit if invalid
 4. Otherwise allows form submission
-

`handleSignupSubmit(event)`

Comprehensive validation before signup submission.

What it does:

1. Gets all form values
2. Runs all validators as safety check
3. Updates feedback elements
4. Checks for any errors
5. If non-college email AND not acknowledged:
 - Prevents form submission
 - Shows non-college modal
 - Waits for user confirmation
6. Otherwise allows submission

Example HTML:

```
<form onsubmit="return handleSignupSubmit(event)">  
  <!-- form fields -->  
</form>
```

Page Initialization

Validation is typically initialized on page load:

```
document.addEventListener('DOMContentLoaded', () => {  
  // Attach real-time validation listeners  
  ['fullname', 'email', 'password', 'confirm-password', 'phone', 'address']  
    .forEach(field => {  
      document.getElementById(field)?.addEventListener('input', updateSubmitButton);  
    });  
});
```

toggle_modes.js

File Location: `src/js/toggle_modes.js`

Purpose: Light/Dark mode theme switching

Dependencies: None

Used by: All pages that need theme toggling

ThemeManager Class

Constructor

```
class ThemeManager {
  constructor() {
    this.theme = localStorage.getItem('theme') || 'light';
    this.sunIcon = '☀';
    this.moonIcon = '🌙';
    this.init();
    this.updateThemeIcon();
  }
}
```

Initialization:

- Loads saved theme from localStorage
 - Defaults to 'light' if first visit
 - Initializes event listeners
 - Updates button icon
-

init()

Initializes theme system.

```
init() {
  this.applyTheme(this.theme);
  this.setupEventListeners();
}
```

What it does:

1. Applies saved theme to page
 2. Attaches click handler to toggle button
-

setupEventListeners()

Attaches click handler to theme toggle button.

```
setupEventListeners() {
  const toggleBtnIndex = document.getElementById('themeToggleBtn');
  if (toggleBtnIndex) {
    toggleBtnIndex.addEventListener('click', () => this.toggle());
  }
}
```

updateThemeIcon()

Updates button icon to show current theme.

```
updateThemeIcon() {
  const isDark = this.theme === 'dark';
  const icon = document.getElementById('themeIcon');
  if (icon) {
    icon.className = isDark ? 'fa-solid fa-sun' : 'fa-solid fa-moon';
  }
}
```

Logic:

- Dark mode = show sun icon (click to switch to light)
 - Light mode = show moon icon (click to switch to dark)
-

toggle()

Switches theme and saves preference.

```
toggle() {
  this.theme = this.theme === 'light' ? 'dark' : 'light';
  this.applyTheme(this.theme);
  localStorage.setItem('theme', this.theme);
}
```

toggleBtn()

Toggles theme with icon update (alternate method).

```
toggleBtn() {
  this.theme = this.theme === 'light' ? 'dark' : 'light';
  this.applyTheme(this.theme);
  this.updateThemeIcon();
  localStorage.setItem('theme', this.theme);
}
```

applyTheme(themeName)

Applies theme classes to page.

```
applyTheme(themeName) {
  const body = document.body;
  const btnIndex = document.getElementById('themeToggleBtn');

  if (themeName === 'dark') {
    body.classList.add('dark-mode');
    if (btnIndex) {
      btnIndex.textContent = this.sunIcon + ' Light Mode';
      btnIndex.setAttribute('aria-label', 'Switch to light mode');
    }
  } else {
    body.classList.remove('dark-mode');
    if (btnIndex) {
      btnIndex.textContent = this.moonIcon + ' Dark Mode';
      btnIndex.setAttribute('aria-label', 'Switch to dark mode');
    }
  }
}
```

What it does:

1. Adds or removes 'dark-mode' class on body
2. Updates button text to show opposite mode
3. Sets accessibility label

CSS Usage:

```
/* Light mode (default) */
body { background: beige; }

/* Dark mode */
body.dark-mode {
  background: #313131;
  color: white;
}
```

setTheme(themeName)

Manually set theme to specific value.

```
setTheme(themeName) {
  if (['light', 'dark'].includes(themeName)) {
    this.theme = themeName;
    this.applyTheme(this.theme);
    localStorage.setItem('theme', this.theme);
  }
}
```

Parameters:

- `themeName` - 'light' or 'dark'

getTheme()

Returns current theme.

```
getTheme() {
  return this.theme;
}
```

Returns: 'light' or 'dark'

Module Initialization

```
let themeManager;

if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', () => {
    themeManager = new ThemeManager();
  });
} else {
  themeManager = new ThemeManager();
}
```

Why two cases?

- Script might load after DOM ready (happens on some pages)
 - Handles both early and late script execution
-

both_homepage.js

File Location: `src/js/both_homepage.js`

Purpose: Shared event loading and display for both admin and student dashboards

Dependencies: `modal1.js`

Used by: `admin_homepage.html`, `student_homepage.html`

Event Loading Functions

loadUpcomingEvents()

Fetches and displays upcoming events in grid layout.

What it does:

1. Fetches from `get_upcoming_events.php`
2. For each event, creates event card HTML
3. Includes event photo as background image
4. Shows event name, category, date, description preview
5. Adds "View Details" button

HTML Generated:

```
<div class="event-card-new">
  <div class="event-card-header" style="background-image: url(...)">
    <h3>Event Name</h3>
    <span class="event-category">Sports</span>
    <span class="event-card-date">2024-05-15</span>
  </div>
  <div class="event-card-body">
    <p>Description preview...</p>
  </div>
  <div class="event-card-footer">
    <button onclick="viewEventDetails(5)">View Details</button>
  </div>
</div>
```

Grid Container: #eventsGrid

loadRecentEvents()

Fetches and displays recently completed events.

What it does:

- Similar to loadUpcomingEvents()
- Uses past events instead
- Limited to 4 events

Grid Container: #recentEventsGrid

viewEventDetails(eventId)

Opens event details modal with full information.

Parameters:

- `eventId` (integer) - Event to display

Process:

1. Opens modal with `event_details_modal.html`
2. Fetches event details from `get_event_details.php`
3. Calculates event status (Upcoming/Completed/Today)
4. Shows event photo, metadata, description
5. Shows registration buttons (Participate/Volunteer)
6. Disables volunteer button if no volunteers needed

Event Status Logic:

```
if (eventDate > currentDate) {
  status = "Upcoming";
} else if (eventDate < currentDate) {
  status = "Completed";
} else {
  status = "Today";
}
```

Modal Content:

```

<div class="event-photo">
  
</div>
<div class="event-details">
  <h2>Event Name</h2>
  <div class="event-meta">
    <div class="meta-item">
      <strong>Category</strong>
      <span>Sports</span>
    </div>
    <!-- More meta fields -->
  </div>
  <div class="action-buttons">
    <button onclick="registerForEvent(5, 'participant')">
      ☐ Participate
    </button>
    <button onclick="registerForEvent(5, 'volunteer')">
      ☐ Volunteer
    </button>
  </div>
</div>

```

switchMainSection(sectionName)

Toggles between "Home" and "Dashboard" tabs.

Parameters:

- `sectionName` - 'home' or 'dashboard'

What it does:

1. Shows/hides `#homeSection` and `#dashboardSection`
2. Updates active tab styling
3. Sets accessibility attributes (aria-selected, aria-hidden)

DOM Manipulation:

```

if (sectionName === 'dashboard') {
  homeSection.classList.remove('active');
  dashboardSection.classList.add('active');
  // Updates button states and accessibility
} else {
  // Reverse
}

```

loadCurrentUserProfile()

Loads and displays user profile photo.

What it does:

1. Fetches user info from `get_current_user.php`
2. Updates profile image `#userProfilePic`
3. Sets src to user's photo path

homepage_only.js

File Location: `src/js/homepage_only.js`

Purpose: Event loading for public landing page (non-logged-in users)

Dependencies: `modal1.js`

Used by: `index.php` (public homepage)

Public Event Functions

loadUpcomingEventsforOther()

Loads upcoming events for public visitors.

What it does:

- Similar to `loadUpcomingEvents()` in `both_homepage.js`
 - Path is different: `../php/` instead of `../../php/`
 - Displays in `#eventsGrid`
-

loadPopularEvents()

Loads popular/trending events for public visitors.

What it does:

- Fetches from `get_popular_events.php`
 - Shows most popular events (sorted by participants)
 - Displays in `#popularEventsGrid`
-

viewEventDetails(eventId)

Opens event detail modal - but redirects to login.

What it does:

- For public users, clicking event details redirects to login
 - Shows: `window.location.href = 'login.html'`
 - Forces user to login before seeing full details
-

Page Initialization

```
document.addEventListener('DOMContentLoaded', function() {  
    loadUpcomingEventsforOther();  
    loadPopularEvents();  
});
```

Loads events when page ready.

admin_dashboard.js

File Location: `src/js/admin_dashboard.js`

Purpose: Admin dashboard interactivity for managing all admin functions

Dependencies: `modal1.js`, `history_handler.js`

Used by: `admin_homepage.html`

Modal Management

currentListType

Module variable tracking which list is currently displayed.

```
let currentListType = '';  
let currentListData = [];
```

openAdminListModal(type)

Opens generic list modal for different admin functions.

Parameters:

- `type` - `'edit_event'`, `'total_events'`, `'active_users'`, `'edit_users'`, `'complaints'`, `'feedback'`, `'items_reported'`, `'event_requests'`

What it does:

1. Sets `currentListType`

- 2. Opens `admin_list_modal.html`
- 3. Calls `loadListData(type)`

loadListData(type)

Fetches appropriate data based on list type.

URLs by Type:

type	URL
edit_event, total_events	get_all_events.php
active_users, edit_users	get_all_users.php
complaints, feedback	get_all_events.php
items_reported	get_reported_items.php
event_requests	get_event_requests.php

Process:

- 1. Shows "Loading..." message
- 2. Fetches data from appropriate endpoint
- 3. Stores in `currentListData`
- 4. Calls `renderAdminList(data, type)`

renderAdminList(data, type)

Renders appropriate HTML for different list types.

For Events:

```
<div class="list-info">
  <h4>Event Name</h4>
  <p>2024-05-15 | Sports | Location</p>
</div>
<button onclick="viewEventInList(5, 'edit_event')">Edit</button>
```

For Users:

```
<div class="list-info">
  <h4>User Name</h4>
  <p>email@kcc.edu.np | admin/student</p>
</div>
<button onclick="viewUserInList(3, 'edit_users')">Edit</button>
```

For Complaints:

- Shows complaint count for each event
- Disables "View" button if no complaints

For Feedback:

- Shows feedback count for each event
- Disables "View" button if no feedback

For Reported Items:

```
<div class="list-info">
  <h4>Item Name</h4>
  <p>🔍 Lost • 2024-05-15 | Location</p>
</div>
<button onclick="viewReportedItem(10, 'lost')">View Details</button>
```

Event Management Functions

viewEventInList(eventId, type)

Opens event details or editing modal.

Parameters:

- `eventId` - Event to view/edit
- `type` - 'edit_event' or 'view'

If 'edit_event':

- Opens `edit_event.html` modal
- Pre-populates with event data

If 'view':

- Opens event details (read-only)
-

viewEventComplaints(eventId)

Shows all complaints for specific event.

Parameters:

- `eventId` - Event to view complaints for

What it does:

1. Fetches complaints from `get_event_complaints.php`
 2. Displays complaint list
 3. Each complaint has "View" button
 4. Opens complaint detail modal on click
-

viewEventFeedback(eventId)

Shows all feedback for specific event.

Parameters:

- `eventId` - Event to view feedback for

What it does:

1. Fetches feedback from `get_event_feedback.php`
 2. Shows feedback with ratings (1-5 stars)
 3. Displays feedback text
-

User Management

viewUserInList(userId, type)

Opens user details or edit modal.

Parameters:

- `userId` - User to view/edit
 - `type` - 'edit_users' or 'view'
-

Item Management

viewReportedItem(itemId, itemType)

Shows details of lost/found item.

Parameters:

- `itemId` - Item ID
- `itemType` - 'lost' or 'found'

What it does:

1. Fetches item details
 2. Shows item photo, description, location
 3. May show action buttons (mark as found, etc.)
-

Event Request Management

viewEventRequest(requestId)

Shows student-requested event details.

Parameters:

- requestId - Request to review

What it does:

- Fetches request details
- Shows proposed event information
- Shows student who requested it
- Approve/Reject buttons

student_dashboard.js

File Location: src/js/student_dashboard.js

Purpose: Student dashboard interactivity

Dependencies: modal1.js, history_handler.js

Used by: student_homepage.html

Similar to Admin Dashboard

Most functions are parallel to admin_dashboard.js but for student-specific operations:

openStudentListModal(type)

Opens list modal for student views.

Possible types:

- 'participated' - Events participated in
- 'registrations' - My registrations
- 'complaints' - Complaints filed
- 'feedback' - Feedback submitted
- 'items_reported' - Items reported
- 'admin_replies' - Admin responses

loadStudentListData(type)

Fetches student-specific data.

URLs:

type	URL
participated	get_participated_events.php
registrations	get_my_registrations.php
complaints	get_my_complaints.php
feedback	get_my_feedback.php
items_reported	get_reported_items.php
admin_replies	get_admin_replies.php

viewEventDetailOnly(eventId)

Views event details without registration buttons (for viewing only).

history_handler.js

File Location: src/js/history_handler.js

Purpose: Manages history modals showing past requests, complaints, feedback, items

Dependencies: modal1.js

Used by: history_modal.html

History Types Configuration

history_types Object

Defines handlers for different history types.

```
const history_types = {
  eventRequest: { ... },
  complaint: { ... },
  feedback: { ... },
  reportedItem: { ... }
}
```

Each type has:

- title - Modal title
- url - PHP endpoint to fetch data
- searchKey - Field to search when filtering
- renderRow(item) - HTML for list item
- renderDetail(item) - HTML for detail view

Event Request History

eventRequest Handler

renderRow:

```
<div class="history-item">
  <h4>Event Name</h4>
  <p>Date | Category</p>
  <span class="history-status status-pending">Pending</span>
  <button onclick="openHistoryDetail('eventRequest', 25)">View</button>
</div>
```

Status Classes:

- status-pending - Awaiting admin decision
- status-approved - Accepted by admin
- status-rejected - Rejected by admin

renderDetail: Shows full event request info:

- Event name, category, date, time
- Venue, volunteers needed
- Status, submitted date
- Full description

Complaint History

complaint Handler

renderRow:

```
<div class="history-item">
  <h4>Complaint Subject</h4>
  <p>Event Name | Event Date</p>
  <span class="history-status status-pending">Pending</span>
  <button onclick="openHistoryDetail('complaint', 5)">View</button>
</div>
```

Status:

- "Pending" if no admin reply

- "Replied" if admin has responded

renderDetail: Shows:

- Complaint subject, text
 - Event info, submitted date
 - Admin reply (if exists) with admin name and reply date
-

Feedback History

feedback Handler

renderRow: Shows past event with student's feedback rating and text.

Reported Item History

reportedItem Handler

renderRow: Shows lost/found item with status.

Status Indicators:

-  Lost Item
 -  Found Item
-

Core Functions

openHistoryModal(historyType)

Opens history modal for specific type.

Parameters:

- `historyType` - 'eventRequest', 'complaint', 'feedback', 'reportedItem'

What it does:

1. Opens `history_modal.html`
 2. Fetches data from type's URL
 3. Renders list of items
 4. Attaches event listeners
-

openHistoryDetail(historyType, itemId)

Opens detail view for specific item.

Parameters:

- `historyType` - Type of history item
- `itemId` - Specific item ID

What it does:

1. Gets item data from `currentHistoryData`
 2. Calls `renderDetail()` for that type
 3. Displays detail view in modal
-

searchHistory(query)

Filters history items by search query.

What it does:

1. Takes search text
 2. Filters `currentHistoryData` by `searchKey`
 3. Re-renders filtered list
-

Page Initialization

History is loaded when modal opens via `openHistoryModal()` .

Quick Function Reference

Function	File	Purpose
openModal()	modal1.js	Open dynamic modal
closeModal()	modal1.js	Close modal
validateEmail()	valid1.js	Check email format
validatePassword()	valid1.js	Check password strength
setFeedback()	valid1.js	Show validation message
themeManager.toggle()	toggle_modes.js	Switch theme
loadUpcomingEvents()	both_homepage.js	Load upcoming event cards
viewEventDetails()	both_homepage.js	Open event detail modal
switchMainSection()	both_homepage.js	Switch tabs
openAdminListModal()	admin_dashboard.js	Open admin list
loadListData()	admin_dashboard.js	Fetch admin data
renderAdminList()	admin_dashboard.js	Render admin list
openStudentListModal()	student_dashboard.js	Open student list
openHistoryModal()	history_handler.js	Open history view
openHistoryDetail()	history_handler.js	View history item detail

Data Flow Examples

1. Event Discovery Flow

```
User visits homepage
→ homepage_only.js loads
→ loadUpcomingEventsforOther() runs
→ Fetches get_upcoming_events.php
→ Creates event cards in #eventsGrid
→ User clicks "View Details"
→ viewEventDetails(eventId) called
→ Redirects to login (non-logged-in)
```

2. Event Registration Flow

```
Logged-in student views event
→ viewEventDetails() opens modal
→ Shows event + Register buttons
→ Student clicks "Participate"
→ registerForEvent(eventId, 'participant') sends AJAX
→ POST to register_event.php
→ Success: shows confirmation
→ Modal closes or shows "Already registered"
```

3. Admin Review Flow

```
Admin opens dashboard
  → openAdminListModal('complaints') called
    → loadListData() fetches get_event_complaints.php
      → renderAdminList() shows events with complaint counts
        → Admin clicks event with 4 complaints
          → viewEventComplaints() opens complaint list
            → Shows individual complaints
              → Admin clicks complaint
                → Opens complaint detail + reply form
                  → Admin submits reply
                    → Calls reply_complaint.php
                      → Success: reply stored, shown to student
```

Document Last Updated: May 22, 2026

Total JavaScript Files Documented: 8

Total Functions Documented: 50+